

EXERCISE 7 - Area of the Mandelbrot Set

Files : area_serial.c, area_parallel.c

Compile / run

```
gcc -O2 -fopenmp area_serial.c -o area_serial && ./area_serial
```

```
gcc -O2 -fopenmp area_parallel.c -o area_parallel
```

```
OMP_NUM_THREADS=4 ./area_parallel # mode 0 (default): block mapping
```

```
OMP_NUM_THREADS=4 ./area_parallel 1 # mode 1: cyclic mapping
```

```
OMP_NUM_THREADS=4 ./area_parallel 2 # mode 2: schedule(dynamic,10)
```

```
Exercise 07 - Mandelbrot area

$ gcc -O2 -fopenmp area_serial.c -o area_serial
$ gcc -O2 -fopenmp area_parallel.c -o area_parallel
$ ./area_serial
Serial : Area of Mandelbrot set = 1.50928453 +/- 0.00075464 time = 15.958 s
Correct answer should be around 1.510659
$ OMP_NUM_THREADS=1 ./area_parallel
Parallel : threads=1 mapping=block (manual) Area = 1.50928453 +/- 0.00075464 time = 16.238 s
$ OMP_NUM_THREADS=2 ./area_parallel
Parallel : threads=2 mapping=block (manual) Area = 1.50928453 +/- 0.00075464 time = 13.866 s
$ OMP_NUM_THREADS=3 ./area_parallel
Parallel : threads=3 mapping=block (manual) Area = 1.50928453 +/- 0.00075464 time = 9.904 s
$ OMP_NUM_THREADS=4 ./area_parallel
Parallel : threads=4 mapping=block (manual) Area = 1.50928453 +/- 0.00075464 time = 7.684 s

# extra exercise: different mappings of iterations to threads (4 threads)
$ OMP_NUM_THREADS=4 ./area_parallel 1
Parallel : threads=4 mapping=cyclic (manual) Area = 1.50928453 +/- 0.00075464 time = 4.228 s
$ OMP_NUM_THREADS=4 ./area_parallel 2
Parallel : threads=4 mapping=schedule(dynamic,10) Area = 1.50928453 +/- 0.00075464 time = 3.963 s
```

Parallelisation

```
#pragma omp parallel default(none) shared(mode) reduction(+:numoutside)
```

```
{
```

```
    int nthreads = omp_get_num_threads();    // library routines
```

```
    int tid = omp_get_thread_num();
```

```
    int lo = (tid * NPOINTS) / nthreads;    // equal block of rows
```

```

int hi = ((tid+1) * NPOINTS) / nthreads;    // for each thread
for (int i = lo; i < hi; i++)
    numoutside += do_row(i);                // z = z*z + c loop
}

```

- * The parallel region is started BEFORE the main loop.
- * z, c, iter, ztemp, j are declared inside the region -> PRIVATE.
- * numoutside is a REDUCTION - each thread counts its own escapes and OpenMP sums them at the end.
- * The outer loop is distributed by hand with `omp_get_num_threads()` / `omp_get_thread_num()` so every thread gets the same number of rows.

Results

threads	mapping	area	time (s)	speed-up
-----	-----	-----	-----	-----
serial	-	1.50928453	15.96	1.0
1	block	1.50928453	16.24	1.0
2	block	1.50928453	13.87	1.2
3	block	1.50928453	9.90	1.6
4	block	1.50928453	7.68	2.1
4	cyclic	1.50928453	4.23	3.8
4	dynamic,10	1.50928453	3.96	4.0

- * The area is IDENTICAL for every thread count (1.50928453 +/- 0.00075).
- * Extra exercise: block mapping scales poorly because the work per row is

very uneven - points inside the set run all 2000 iterations, points far outside escape immediately, so the threads holding the middle rows do most of the work. Cyclic or dynamic mapping spreads heavy and light rows across all threads and gives near-perfect speed-up on 4 threads.